Geth RPC API 시스템 상세 분석

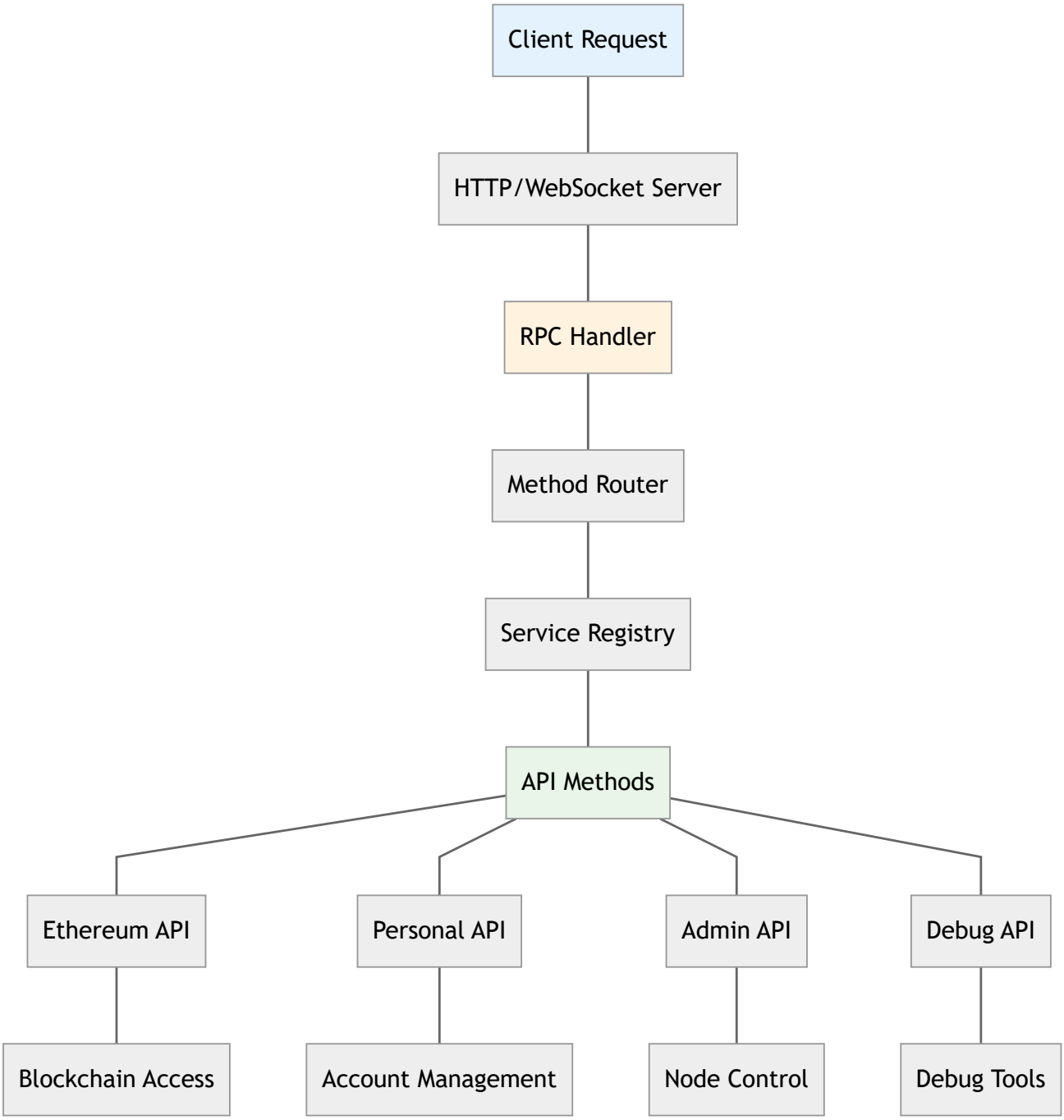
Ethereum 노드와 외부 애플리케이션 간의 통신을 담당하는 RPC API 시스템의 구조와 동작 원리를 분석한 문서입니다.

> **분석 범위:** JSON-RPC 2.0부터 WebSocket 이벤트까지

> **주요 소스:** `rpc/` , `internal/ethapi/` , `node/`

> **핵심 개념:** JSON-RPC, HTTP/WS 서버, 메서드 라우팅, 구독

🚀 RPC 시스템 전체 아키텍처



📋 RPC 서버 구조

1. 서버 초기화

```
// rpc/server.go
type Server struct {
```

```
services serviceRegistry

idgen  func() ID

run  int32

codecs mapset.Set

}



// 서비스 등록소

type serviceRegistry map[string]*service



type service struct {

name  string

typ reflect.Type

callbacks callbacks

subscriptions subscriptions

}

```// H

TTP 서버 설정

type HTTPConfig struct {

Modules []string // 활성화할 API 모듈들

CorsAllowedOrigins []string // CORS 허용 오리진

Vhosts []string // 가상 호스트

prefix string // URL 프리픽스

jwtSecret []byte // JWT 시크릿
```
```

```
}


// WebSocket 서버 설정
type WSConfig struct {
Origins []string // 허용된 오리진들
Modules []string // 활성화할 API 모듈들
prefix string // URL 프리픽스
}


// RPC 서버 시작
func (n *Node) startRPC() error {
if err := n.startHTTP(); err != nil {
return err
}
if err := n.startWS(); err != nil {
n.stopHTTP()
return err
}
if err := n.startIPC(); err != nil {
n.stopHTTP()
n.stopWS()
return err
}
```

```
return nil
}
```

2. 메서드 등록 및 라우팅

```
// 서비스 등록
func (s *Server) RegisterName(name string, receiver interface{}) error {
return s.services.registerName(name, receiver)
}

// 메서드 등록
func (r *serviceRegistry) registerName(name string, rcvr interface{}) error {
rcvrVal := reflect.ValueOf(rcvr)
if name == "" {
return fmt.Errorf("no service name for type %s", rcvrVal.Type().String())
}
callbacks := suitableCallbacks(rcvrVal)
if len(callbacks) == 0 {
return fmt.Errorf("service %T doesn't have any suitable methods/subscriptions t
}
(*r)[name] = &service{
name: name,
typ: rcvrVal.Type(),
```

```
callbacks: callbacks,

subscriptions: suitableSubscriptions(rcvrVal),

}

return  nil

}



// 적합한 콜백 메서드 찾기

func  suitableCallbacks(receiver reflect.Value) map[string]*callback {

typ := receiver.Type()

callbacks := make(map[string]*callback)

for  m := 0;  m < typ.NumMethod();  m++ {

method := typ.Method(m)

if  method.PkgPath != "" { // 메서드가 exported되지 않음

continue

}

cb := &callback{

receiver: receiver,

method: method.Func,

errPos: -1,

isSubscribe: isPubSub(method),

}

if  cb.isSubscribe {

cb.subscribeMethodName = formatName(method.Name[9:]) // "Subscribe" 제거
```

```
}

callbacks[formatName(method.Name)] = cb

}

return  callbacks

}
```

3. 요청 처리

```
// JSON-RPC 요청 구조체

type jsonrpcMessage struct {

Version  string  `json:"jsonrpc,omitempty"`

ID json.RawMessage `json:"id,omitempty"`

Method  string  `json:"method,omitempty"`

Params json.RawMessage `json:"params,omitempty"`

Error *jsonError `json:"error,omitempty"`

Result json.RawMessage `json:"result,omitempty"`

}


// 요청 처리

func (s *Server) handle(ctx context.Context, codec ServerCodec, req *jsonrpcMess

if  req.isNotification() {

s.exec(ctx, codec, req)

return  nil  // 알림은 응답하지 않음
```

```
}
return s.exec(ctx, codec, req)
}

// 메서드 실행
func (s *Server) exec(ctx context.Context, codec ServerCodec, req *jsonrpcMessag
var response *jsonrpcMessage
if req.isPubSub() {
response = s.execSubscription(ctx, codec, req)
} else {
response = s.execMethod(ctx, codec, req)
}
return response
}

// 일반 메서드 실행
func (s *Server) execMethod(ctx context.Context, codec ServerCodec, req *jsonrpc
// 메서드 찾기
callb := s.services.callback(req.Method)
if callb == nil {
return req.errorResponse(&methodNotFoundError{method: req.Method})
}
// 파라미터 파싱
```

```
args, err := parsePositionalArguments(req.Params, callb.argTypes)
if err != nil {
return req.errorResponse(&invalidParamsError{err.Error()})
}
// 메서드 호출
start := time.Now()
answer := callb.call(ctx, req.Method, args)
duration := time.Since(start)
// 메트릭 업데이트
rpcRequestGauge.Inc(1)
rpcRequestTimer.Update(duration)
if answer.Error != nil {
return req.errorResponse(answer.Error)
}
return req.response(answer.Result)
}
```

🌐 HTTP 서버

1. HTTP 핸들러

```
// node/rpcstack.go
type httpServer struct {
```

```
log log.Logger
timeouts rpc.HTTPTimeouts
mux http.ServeMux
mu sync.Mutex
server *http.Server
listener net.Listener
}


// HTTP 요청 처리
func (h *httpServer) ServeHTTP(w http.ResponseWriter, r *http.Request) {
// CORS 헤더 설정
if  origin := r.Header.Get("Origin");  origin != "" {
w.Header().Set("Access-Control-Allow-Origin", origin)
w.Header().Set("Access-Control-Allow-Credentials", "true")
}
// 프리플라이트 요청 처리
if  r.Method == "OPTIONS" {
w.Header().Set("Access-Control-Allow-Methods", "POST, GET, OPTIONS")
w.Header().Set("Access-Control-Allow-Headers", "Content-Type")
return
}
// Content-Type 확인
if  r.Header.Get("Content-Type") != "application/json" {
```

```
http.Error(w, "Content-Type must be application/json", http.StatusUnsupportedMed
return
}
// 요청 크기 제한
r.Body = http.MaxBytesReader(w, r.Body, maxRequestContentLength)
// RPC 서버로 전달
h.mux.ServeHTTP(w, r)
}


// RPC 핸들러 등록
func newHTTPHandler(stack *rpc.Server, cors []string, vhosts []string, jwtSecre
// 기본 핸들러 생성
h := stack.HTTPHandler()
// JWT 인증 미들웨어
if len(jwtSecret) != 0 {
h = newJWTHandler(h, jwtSecret)
}
// 가상 호스트 검증
if len(vhosts) > 0 {
h = newVHostHandler(vhosts, h)
}
// CORS 미들웨어
if len(cors) > 0 {
```

```
h = newCorsHandler(h, cors)
}
return h
}
```

2. CORS 및 보안

```
// CORS 핸들러
func newCorsHandler(srv http.Handler, allowedOrigins []string) http.Handler {
// 와일드카드 처리
hasWildcard := false
for _, origin := range allowedOrigins {
if origin == "*" {
hasWildcard = true
break
}
}
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
origin := r.Header.Get("Origin")
if origin != "" && (hasWildcard || contains(allowedOrigins, origin)) {
w.Header().Set("Access-Control-Allow-Origin", origin)
w.Header().Set("Access-Control-Allow-Credentials", "true")
```

```
}
if r.Method == "OPTIONS" {
w.Header().Set("Access-Control-Allow-Methods", "POST, GET, OPTIONS")
w.Header().Set("Access-Control-Allow-Headers", "Content-Type")
w.WriteHeader(http.StatusOK)
return
}
srv.ServeHTTP(w, r)
})
}


// JWT 인증 핸들러
func newJWTHandler(keyfile string, next http.Handler) http.Handler {
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
// Authorization 헤더 확인
auth := r.Header.Get("Authorization")
if auth == "" {
http.Error(w, "missing token", http.StatusUnauthorized)
return
}
// Bearer 토큰 추출
if !strings.HasPrefix(auth, "Bearer ") {
http.Error(w, "invalid token format", http.StatusUnauthorized)
```

```
return
}
token := auth[7:]
// JWT 검증
if !validateJWT(token, keyfile) {
http.Error(w, "invalid token", http.StatusUnauthorized)
return
}
next.ServeHTTP(w, r)
})
}
```

🔌 WebSocket 서버

1. WebSocket 연결 관리

```
// rpc/websocket.go
type websocketCodec struct {
conn *websocket.Conn
wmu sync.Mutex // 쓰기 뮤텍스
// 구독 관리
mu sync.Mutex
subs  map[string]*notificationSub
```

```go
}


// WebSocket 핸들러
func wsHandler(allowedOrigins []string, srv *Server) http.Handler {
upgrader := websocket.Upgrader{
ReadBufferSize: 1024,
WriteBufferSize: 1024,
CheckOrigin: func(r *http.Request) bool {
origin := r.Header.Get("Origin")
if origin == "" {
return true
}
for _, allowed := range allowedOrigins {
if allowed == "*" || allowed == origin {
return true
}
}
return false
},
}
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
conn, err := upgrader.Upgrade(w, r, nil)
if err != nil {
```

```
log.Debug("WebSocket upgrade failed", "err", err)

return

}

codec := newWebsocketCodec(conn)

defer codec.close()

srv.ServeCodec(codec, 0)

})

}



// 메시지 읽기

func (wc *websocketCodec) readLoop(ctx context.Context, msgCh chan<- *jsonrpcMe

defer close(msgCh)

for {

select {

case <-ctx.Done():

return

default:

}

_, message, err := wc.conn.ReadMessage()

if err != nil {

if !websocket.IsCloseError(err, websocket.CloseGoingAway, websocket.CloseNormalC

log.Debug("WebSocket read error", "err", err)

}
```

```
return
}
var msg jsonrpcMessage
if err := json.Unmarshal(message, &msg); err != nil {
log.Debug("Invalid JSON-RPC message", "err", err)
continue
}
select {
case msgCh <- &msg:
case <-ctx.Done():
return
}
}
}
```

2. 구독 시스템

```
// 구독 관리자
type SubscriptionManager struct {
mu sync.RWMutex
subscriptions map[ID]*Subscription
unsubscribe chan *Subscription
}
```

```go
// 구독 구조체
type Subscription struct {
ID ID
namespace  string
typ reflect.Type
created time.Time
logsCrit ethereum.FilterQuery // 로그 필터
installed  chan  struct{} // 설치 완료 신호
err  chan  error  // 에러 채널
}


// 구독 생성
func (api *FilterAPI) NewPendingTransactionFilter() rpc.ID {
var (
pendingTxs = make(chan []common.Hash)
pendingTxSub = api.events.SubscribePendingTxs(pendingTxs)
)
api.filtersMu.Lock()
api.filters[pendingTxSub.ID] = &filter{
typ: PendingTransactionsSubscription,
deadline: time.NewTimer(timeout),
hashes: make([]common.Hash, 0),
```

```
s: pendingTxSub,
}
api.filtersMu.Unlock()
go func() {
defer pendingTxSub.Unsubscribe()
for {
select {
case ph := <-pendingTxs:
api.filtersMu.Lock()
if f, found := api.filters[pendingTxSub.ID]; found {
f.hashes = append(f.hashes, ph...)
}
api.filtersMu.Unlock()
case <-pendingTxSub.Err():
return
}
}
}()
return pendingTxSub.ID
}


// 로그 구독
func (api *FilterAPI) NewFilter(crit FilterCriteria) (rpc.ID, error) {
```

```
logs := make(chan []*types.Log)

logsSub, err := api.events.SubscribeLogs(ethereum.FilterQuery(crit), logs)

if err != nil {

return "", err

}

api.filtersMu.Lock()

api.filters[logsSub.ID] = &filter{

typ: LogsSubscription,

crit: crit,

deadline: time.NewTimer(timeout),

logs: make([]*types.Log, 0),

s: logsSub,

}

api.filtersMu.Unlock()

go func() {

defer logsSub.Unsubscribe()

for {

select {

case l := <-logs:

api.filtersMu.Lock()

if f, found := api.filters[logsSub.ID]; found {

f.logs = append(f.logs, l...)

}
```

```
api.filtersMu.Unlock()

case <-logsSub.Err():

return

}

}

}()

return  logsSub.ID, nil

}
```

📡 API 모듈들

1. Ethereum API

```
// internal/ethapi/api.go

type EthereumAPI struct {

b Backend

}


// 잔액 조회

func (s *EthereumAPI) GetBalance(ctx context.Context, address common.Address, bl

state, _, err := s.b.StateAndHeaderByNumberOrHash(ctx, blockNrOrHash)

if  state == nil || err != nil {

return  nil, err
```

```
}

return (*hexutil.Big)(state.GetBalance(address)), state.Error()

}



// 트랜잭션 전송

func (s *EthereumAPI) SendTransaction(ctx context.Context, args TransactionArgs)

// 계정 잠금 해제 확인

if  args.From == nil {

return common.Hash{}, fmt.Errorf("missing from address")

}

// 트랜잭션 생성

tx, err := s.createTransaction(ctx, args)

if  err != nil {

return common.Hash{}, err

}

// 서명

signed, err := s.sign(*args.From, tx)

if  err != nil {

return common.Hash{}, err

}

// 트랜잭션 풀에 추가

if  err := s.b.SendTx(ctx, signed);  err != nil {

return common.Hash{}, err
```

```
}

return signed.Hash(), nil

}


// 블록 조회

func (s *EthereumAPI) GetBlockByNumber(ctx context.Context, number rpc.BlockNumb

block, err := s.b.BlockByNumber(ctx, number)

if block != nil && err == nil {

response, err := s.rpcMarshalBlock(ctx, block, true, fullTx)

if err == nil && number == rpc.PendingBlockNumber {

// 펜딩 블록의 경우 해시와 논스를 nil로 설정

for _, field := range []string{"hash", "nonce", "miner"} {

response[field] = nil

}

}

return response, err

}

return nil, err

}
```

2. Personal API

```
// internal/ethapi/api.go
```

```go
type PersonalAccountAPI struct {
am *accounts.Manager
nonceLock *AddrLocker
b Backend
}


// 새 계정 생성
func (s *PersonalAccountAPI) NewAccount(password  string) (common.Address, error
ks, err := fetchKeystore(s.am)
if  err != nil {
return common.Address{}, err
}
acc, err := ks.NewAccount(password)
if  err == nil {
log.Info("Your new key was generated", "address", acc.Address)
log.Warn("Please backup your key file!", "path", acc.URL.Path)
log.Warn("Please remember your password!")
return  acc.Address, nil
}
return common.Address{}, err
}


// 계정 잠금 해제
```

```go
func (s *PersonalAccountAPI) UnlockAccount(ctx context.Context, addr common.Addr
// 기본 잠금 해제 시간 (5분)
const max = uint64(300)
var d time.Duration
if duration == nil {
d = 300 * time.Second
} else if *duration > max {
return false, errors.New("unlock duration too large")
} else {
d = time.Duration(*duration) * time.Second
}
ks, err := fetchKeystore(s.am)
if err != nil {
return false, err
}
err = ks.TimedUnlock(accounts.Account{Address: addr}, password, d)
if err != nil {
log.Warn("Failed account unlock attempt", "address", addr, "err", err)
}
return err == nil, err
}
```

3. Admin API

```go
// node/api.go
type adminAPI struct {
node *Node
}

// 피어 정보 조회
func (api *adminAPI) Peers() ([]*p2p.PeerInfo, error) {
server := api.node.Server()
if server == nil {
return nil, ErrNodeStopped
}
return server.PeersInfo(), nil
}

// 피어 추가
func (api *adminAPI) AddPeer(url string) (bool, error) {
server := api.node.Server()
if server == nil {
return false, ErrNodeStopped
}
node, err := enode.Parse(enode.ValidSchemes, url)
```

```
if err != nil {
return false, fmt.Errorf("invalid enode: %v", err)
}
server.AddPeer(node)
return true, nil
}

// 노드 정보
func (api *adminAPI) NodeInfo() (*p2p.NodeInfo, error) {
server := api.node.Server()
if server == nil {
return nil, ErrNodeStopped
}
return server.NodeInfo(), nil
}
```

📊 성능 최적화 및 모니터링

1. 요청 제한 및 캐싱

```
// 요청 속도 제한
type rateLimiter struct {
requests map[string]*rate.Limiter
```

```
mu sync.RWMutex

rate rate.Limit

burst  int

}


func (rl *rateLimiter) allow(ip  string) bool {

rl.mu.RLock()

limiter, exists := rl.requests[ip]

rl.mu.RUnlock()

if !exists {

rl.mu.Lock()

limiter = rate.NewLimiter(rl.rate, rl.burst)

rl.requests[ip] = limiter

rl.mu.Unlock()

}

return  limiter.Allow()

}


// 응답 캐싱

type responseCache struct {

cache  map[string]*cacheEntry

mu sync.RWMutex

ttl time.Duration
```

```
}


type cacheEntry struct {
response  interface{}
timestamp time.Time
}


func (rc *responseCache) get(key  string) (interface{}, bool) {
rc.mu.RLock()
defer  rc.mu.RUnlock()
entry, exists := rc.cache[key]
if !exists || time.Since(entry.timestamp) > rc.ttl {
return  nil, false
}
return  entry.response, true
}
```

2. 메트릭 수집

```
// RPC 메트릭
var (
rpcRequestGauge = metrics.NewRegisteredGauge("rpc/requests", nil)
rpcRequestTimer = metrics.NewRegisteredTimer("rpc/duration/all", nil)
```

```
rpcSuccessTimer = metrics.NewRegisteredTimer("rpc/duration/successful", nil)

rpcFailureTimer = metrics.NewRegisteredTimer("rpc/duration/failed", nil)

rpcServingTimer = metrics.NewRegisteredTimer("rpc/duration/serving", nil)

// 메서드별 메트릭

rpcMethodTimers = make(map[string]metrics.Timer)

rpcMethodCounters = make(map[string]metrics.Counter)

)



// 메트릭 업데이트

func updateRPCMetrics(method string, duration time.Duration, success bool) {

rpcRequestGauge.Inc(1)

rpcRequestTimer.Update(duration)

if success {

rpcSuccessTimer.Update(duration)

} else {

rpcFailureTimer.Update(duration)

}

// 메서드별 메트릭

if timer, exists := rpcMethodTimers[method]; exists {

timer.Update(duration)

} else {

timer = metrics.NewRegisteredTimer("rpc/duration/"+method, nil)

rpcMethodTimers[method] = timer
```

```
timer.Update(duration)

}

if  counter, exists := rpcMethodCounters[method];  exists {

counter.Inc(1)

} else {

counter = metrics.NewRegisteredCounter("rpc/requests/"+method, nil)

rpcMethodCounters[method] = counter

counter.Inc(1)

}

}
```

이 문서를 통해 이해할 수 있는 것:

- JSON-RPC 2.0 프로토콜의 구현과 메서드 라우팅
- HTTP와 WebSocket 서버의 구조와 보안 메커니즘
- 구독 시스템과 실시간 이벤트 처리
- 다양한 API 모듈들의 기능과 구현
- 성능 최적화를 위한 캐싱과 속도 제한
- 실시간 모니터링을 위한 메트릭 수집

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.